PERTOK: EXPRESSIVE ENCODING AND MODELING OF SYMBOLIC MUSICAL IDEAS AND VARIATIONS

Julian Lenz
Lemonaide Research
Barcelona, ES

Anirudh Mani
Lemonaide Research
Boston, US

ABSTRACT

We introduce *Cadenza*, a new multi-stage generative framework for predicting expressive variations of symbolic musical ideas as well as unconditional generations. To accomplish this we propose a novel MIDI encoding method, *PerTok* (Performance Tokenizer) that captures minute expressive details whilst reducing sequence length up to 59% and vocabulary size up to 95% for polyphonic, monophonic and rhythmic tasks. The proposed framework comprises of two sequential stages: 1) *Composer* and 2) *Performer*. The Composer model is a transformer-based Variational Autoencoder (VAE), with Rotary Positional Embeddings (RoPE) [1] and an autoregressive decoder modified to more effectively integrate the latent codes of the input musical idea. The Performer model is a bidirectional transformer encoder that is separately trained to predict velocities and microtimings on MIDI sequences. Objective and human evaluations demonstrate Cadenza's versatile capability in 1) matching other unconditional state-of-the-art symbolic models in musical quality whilst sounding more expressive, and 2) composing new, expressive ideas that are both stylistically related to the input whilst providing novel ideas to the user. Our framework is designed, researched and implemented with the objective of ethically providing inspiration for musicians.

1. INTRODUCTION

The creative endeavor in present-day music production is inherently complex and multifaceted. However, it can be broadly categorized into distinct phases that include 1) initiation, 2) evolution and development, and 3) completion of musical ideas into a finished musical outcome. Modern generative models have had a major impact in every creative domain, none the least in music creation. MIDI, and therefore Symbolic AI research approaches for single-track MIDI generation are especially applicable to the contemporary music producer. The motivation behind our investigation arises from a gap in this current landscape to facilitate the crucial middle phase of the creative process: absence of a comprehensive, adaptable modeling framework specifically engineered for generating expression variations from a given MIDI file input. Our proposed solution, *Cadenza*, addresses this by focusing on the 'development' phase of music creation while unveiling a framework that is designed for flexibility and efficiency. Cadenza utilises a multi-stage generative process, the *composer* and the *performer*, to create novel ideas and variations while emulating the nuanced performance characteristics that can define a given musical style. We choose to call our framework 'Cadenza', inspired by the improvised musical passage played by soloists, creating new and exciting variations of the original motifs of the piece being performed.

1.1 Encoding Symbolic Music

Alongside transformer-based architectures a number of methods have been proposed to encode, or *tokenize* MIDI files into a discrete sequence of tokens. As transformers suffer from quadratic memory complexity in relation to sequence lengths [2], particular focus is placed on capturing relevant MIDI information whilst minimizing the total number of tokens. Popular tokenizers include REMI [3], TSD [4] and Structured [5], amongst many others. However, these approaches suffer from a common drawback: they rely on singular tokens to denote the position of each note event on an evenly-spaced temporal grid. In comparison to MIDI files, which typically utilise a time resolution of 220 or 440 *ticks-per-quarter* [note], these tokenizers are typically employed with just four intervals per quarter note. This has two negative effects: first, note-values outside this range are immediately *quantized*, such as quarter-triplets, eighth-triplets, quintuplets, and thirty-second notes. In addition, any rhythmic performance attributes, expressed as subtle deviations from the fixed-grid, are immediately lost. As a result, the current state-of-the-art in MIDI tokenizers are unable to accurately capture the full range of rhythmic values and expressive performances.

1.2 Expressive Modeling

In both digital and physical contexts, it is common to divide the act of *composing* music and *performing* it. The composition (or score) contains the raw musical idea, whereas the performance will typically embellish it with additional details, such as varying volumes (velocities in MIDI) and subtle timing deviations. Symbolic datasets can

© Julian Lenz, and Anirudh Mani. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** Julian Lenz, and Anirudh Mani, "PerTok: Expressive Encoding and Modeling of Symbolic Musical Ideas and Variations", in *Proc. of the 25th Int. Society for Music Information Retrieval Conf.,* San Francisco, United States, 2024.

be categorized broadly as:

- **Score**: The sequences contain quantized rhythmic values and minimal/no volume information.
- **Time-Performance**: Dynamics and expressive timing are captured. The performer(s) play without a fixed tempo, resulting in a *time*-based encoding (typically milliseconds), such as [6].
- **Beat-Performance**: Dynamics and expressive timing are captured. The notes are recorded in relation to a fixed tempo, with rhythmic expressivity occurring as deviations from the quantized beats.

Although a substantial quantity of **score** datasets exist, there are significantly fewer in both **performance** categories. As a result, systems such as that proposed in [7], wherein both composition and performance elements are jointly trained and predicted, are limited by this inequality.

A number of recent models have been proposed to exclusively add performance elements, such as RenderingRNN [8] and ScorePerformer [9]. However, they rely on the prediction of *tempo* tokens in alignment with the **Time-Performance** standard. This results in MIDI files that are still rhythmically *quantized*, albeit with varying tempos. We posit that this approach is incompatible with the common production standards of many modern genres that instead rely on fixed tempos.

The framework in Compose & Embellish [10] proposed a system of jointly training Lead-Sheet (score) and Performance models. With a modified REMI [3] tokenization they demonstrated that the lead-sheet model could be pre-trained on a greater quantity of **score** data, and subsequently fine-tuned with the performer, on a smaller **performance** dataset. Similar to prior systems, they quantize rhythms to the nearest 16th position, and instead predict `[Tempo]` for expressive timing. Furthermore, due to the joint-conditioning training method, the compose model can lose certain capabilities from the fine-tuning process as it fits to the smaller performance dataset.

1.3 Generating Variations

A number of models have been proposed to solve variations-adjacent tasks. ThemeTransformer [11] utilises contrastive representation learning in a sequence-to-sequence framework to generate a melody and accompaniment that recurrently incorporates the original theme. The authors of Music FaderNets [12] instead propose a *style-transfer* task, wherein a number of high-level attributes can be applied to transform a polyphonic sequence. Our work most notably builds off of the model proposed in MuseMorphose [13], which uses a novel *in-attention* mechanism in a transformer-based VAE for generating attribute-controlled variations on symbolic data. However, these models are designed to predict long-form sequences (16+ bars) that do not contain any expressive information.

Overall, our key contributions to the field through this work are two-fold. Firstly, in section 2.2 we introduce *PerTok*, a novel MIDI encoding method that captures expressive details with required granularity while maintaining compact sequence lengths and manageable vocabulary sizes. PerTok is implemented with the MidiTok [4] library, released open source and is compatible with any token-based sequential generation model. Secondly, the *Cadenza* framework itself represents a significant leap forward as presented in section 3, integrating the 'Composer' and 'Performer' models into a cohesive architecture that is researched and designed for the domain of AI-assisted music creation. Our framework offers a natural and intuitive way for musicians to create and modify music, which can be tailored to specific stylistic goals. Human evaluations showcase how Cadenza matches other state-of-the-art MIDI models in unconditional score generation quality, creates dynamic variations on input ideas, and sets a new standard for human-like expressive articulations.

2. SYMBOLIC DATA ENCODING

We aim to encode MIDI data in a manner that is both 1) aligned with common audio production use-cases and 2) efficient in the context of transformer-based generative models. While a number of state-of-the-art models such as Anticipatory Music Transformer (AMT) [14], Figaro [15] and Multi-Track Music Machine (MMM) [16] have focused on long-form, multi-track generation, we have observed from our experiences in designing products for contemporary music producers that they more commonly interact with short, single-track files. Furthermore, we observe that a number of tokenization methods such as [9] rely on *tempo* tokens to create a sense of expressive performance, whereas music producers often keep a singular, consistent tempo throughout their composition. Thus, our proposed encoding method is focused on single-track MIDI files in which the expressivity is calculated in relation to a fixed tempo.

2.1 Score & Performance Encoding

To address this, we create separate tokens to model the composition and performance timing elements. More specifically, the macro *timeshift* tokens represent the quantized note locations within a score. Separately, the *microshift* events denote a small adjustment from the quantized location, similar to the subtle timing deviations used by human performers. By separating these events, we are able to maintain reasonable balance between vocabulary size and total sequence length. Furthermore, the differentiation allows for models to be separately trained on the composition and performance tasks respectively. As there is a significant difference in the availability between quantized and performed symbolic datasets, this enables us to feed a far greater quantity of data into the composition-only model.

Initial tests revealed that the common convention of quantizing the composition-level timeshift tokens to 16th notes was leading to a number of musical issues. For example, when quarter- or eighth-note triplets were present

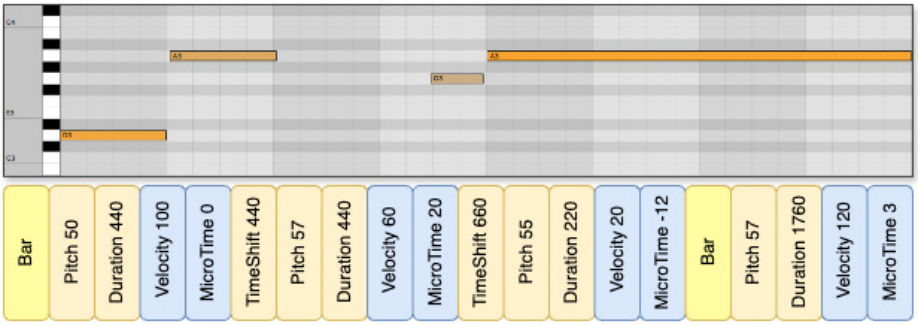


Figure 1. Example of PerTok tokenization on a 2-bar excerpt of a MIDI file. Composition tokens are highlighted in yellow, and performance tokens in blue.

in the input MIDI file, the quantization process was considering them to be 16th notes with large degrees of *microshift*. We addressed this by providing the ability to specify multiple, overlapping quantization grids, such as 16th, quarter-triplets, and 8th-triplets. Thus, the PerTok tokenizer is more adept at capturing the wide variety of rhythmic values that are commonly found in genres such as hip-hop, jazz and salsa (among many others).

2.2 PerTok

Score Tokens: Similar to the MIDI-Like [7] and Structured [5] encoding methods we represent *macro* time changes between notes with **Timeshift** tokens. As MIDI time data is typically expressed as ticks-per-quarter, PerTok allows for multiple overlapping granularities to model a variety of rhythmic values. When encoding the MIDI data, PerTok matches each note's position to the closest possible timeshift value. **Pitch** is denoted as a MIDI pitch value between 0-127, with the capability to limit this range when musically appropriate. **Duration** tokens are used after each new note, to indicate the length of time before a MIDI note-off message is triggered. Notably, PerTok allows for the removal of duration tokens altogether, which helps further reduce sequence lengths when modeling rhythmic instruments.

Performance Tokens: With the addition of performance tokens, we aim to capture the musical subtleties that transform a written score into an expressive performance. **Velocity** tokens denote the strength of the note's attack, a property that is typically used in DAWs to augment timbre and volume characteristics. Although MIDI provides a range of 0 - 127 for velocity values, we allow for a bucketing approach to reduce a given velocity into one of *n* possible values. **Microshift** tokens provide a granular shift from the quantized rhythmic note value. PerTok is provided a maximum microshift value (e.g. 30 MIDI ticks) and a discrete number of possible microshift buckets. For example, *Microshift 15* represents a placement of 15 ticks after the quantized note position, and *Microshift 0* results in the initial quantized value.

In **Table 1** we provide a benchmark of our proposed PerTok encoding against a number of popular MIDI tokenizers. We sampled from 2,000 polyphonic 4- and 8-bar MIDI files that are used in modern audio production environments. For each tokenizer, we use 32 possible velocity buckets. To demonstrate the tradeoff between composition and performance, we initialize one version with 16th-note quantization (thus removing any performance characteristics), and a second (denoted with a -p) version with 440 timeshifts per quarter note. REMI and Structured methodologies use evenly spaced temporal grids to encode the location of each event, whereas PerTok uses a mixture of macro and micro timeshift tokens leading to a 95% reduction in vocabulary size when encoding expressive rhythmic details. We additionally provide a visualization of our encoding method with a sample 2-bar melody in **Figure 1**.

Tokenizer	Vocab. Size	Seq. Length
REMI	273	195
REMI-p	5505	199
Structured	289	216
Structured-p	7265	216
TSD	288	188
TSD-p	7264	192
PerTok	196	134
PerTok-p	259	243
PerTok no-duration	164	80

Table 1. Vocabulary sizes and average sequence lengths for popular tokenizers and our proposed PerTok encoding.

3. MODEL

With the objective of generating expressive, beat-structured variations on an input MIDI file, we present *Cadenza*, a multi-stage VAE with transformer-based components. The framework is designed upon a principle that the *composition* task requires a significant quantity of data and benefits from auto-regressive generation, whereas the patterns of expressive *performance* tokens can be learned with smaller datasets and predicted in a bi-directional manner.

3.1 The Composer

Given an input sequence of tokens $\{x_1, x_2, ..., x_t\}$ in which x_t represents a single token x at index t the **composer** model is designed to auto-regressively predict $\{y_1, y_2, ..., y_t\}$, an output sequence that is musically related yet distinct from the input. We utilise a sequence-to-

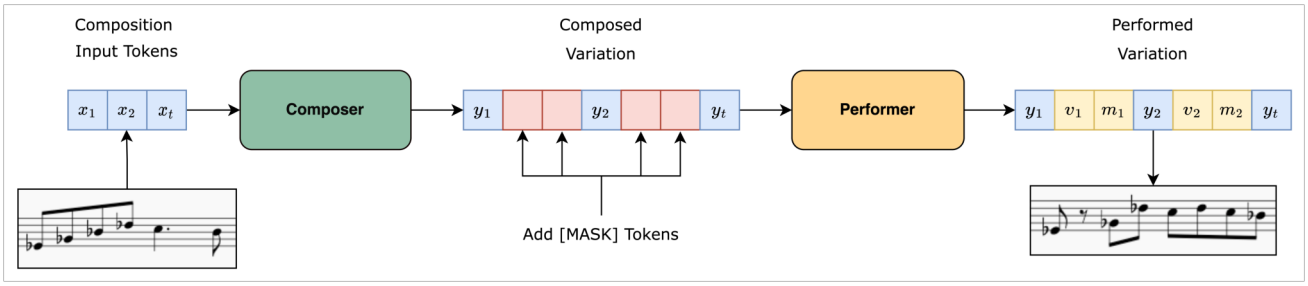


Figure 2. An overview of the multi-stage *Cadenza* architecture.

sequence VAE architecture with in-attention conditioning similar to MuseMorphose [13], enabling the model to learn a compressed, latent representation of short musical ideas within a regularized space. Within each attention mechanism, the query q and key k vectors at timesteps m, n are obtained with RoPE [1] for enhanced positional context:

$$q_m^\top k_n = (R^d_{\Theta,m} W^q x_m)^\top (R^d_{\Theta,n} W^k x_n) \tag{1}$$

Wherein $R^d_{\Theta,m}$ and $R^d_{\Theta,n}$ are the rotary matrices for positions for embedding positional information, W^q and W^k are learnable weight matrices transforming inputs x_m and x_n into the query and key vectors. For additional context we encourage readers to refer the original paper. As music is a deeply temporal phenomenon, the composer benefits from the increased token spatial modelling that is provided by the rotary embeddings.

The encoder is designed to create a latent vector z of the input musical idea which serves as an anchor throughout the decoding process. In alignment with the original transformer [2] we first project the input sequence with a learned embedding space, transforming it into $X \in \mathbb{R}^{d \times t}$ where d is the hidden dimension size. This is then processed through several multi-head self-attention layers. Following a similar approach to [13] we extract the first timestep of the final attention layer output to obtain hidden vector $h \in \mathbb{R}^d$, a contextual representation of the full input sequence.

Following standard VAE methodology [17], the output vector is then processed through two learnable weight matrices $W_\mu \in \mathbb{R}^{d \times d_z}$ and $W_\sigma \in \mathbb{R}^{d \times d_z}$, wherein d_z denotes the size of the latent dimension. This process yields the mean μ and standard deviation σ vectors, encapsulating the latent space distribution parameters. Using the reparameterization trick, we sample ϵ from the Gaussian distribution to obtain $z \in \mathbb{R}^{d_z}$ from our encoder:

$$z = hW_\mu + hW_\sigma \odot \epsilon \tag{2}$$

The encoder's output distribution $q(z|X)$ is aligned to a Gaussian prior $\mathcal{N}(0, 1)$ by the traditional Kullback-Leibler (KL) divergence loss term:

$$D_{KL}(q(z|X)\|p(z)) = -\frac{1}{2}\sum_{k=1}^{d_z}\left(1 + \log(\sigma_k^2) - \mu_k^2 - \sigma_k^2\right) \tag{3}$$

We further modify the equation utilising free bits as proposed by [18], allowing the encoder a degree of unpenalized space defined by λ to learn musical attributes without regularization.

$$\mathcal{L}_{\text{KL}} = \sum_{k=1}^{d_z} \max(\lambda, D_{\text{KL}}(q(z_k|X)||p(z_k))) \tag{4}$$

The decoder is trained to autoregressively predict an output sequence of tokens whilst maintaining a recognizable connection with the input musical idea. Initially, the latent vector z is expanded to the decoder's hidden dimension d via a learnable matrix $W_{\text{pre}} \in \mathbb{R}^{d_z \times d}$. We separately expand the input tokens $v_1, v_2, ..., v_k$ with the same embedding layer used by the encoder.

In the context of the autoregressive VAEs it has been noted that *posterior collapse* is a common issue [19–21], in which a sufficiently powerful decoder can simply ignore the encoder's regularized information, instead relying purely on the previous tokens. We utilise the methodology of Skip-VAE [22] and MuseMorphose [13], integrating the proposed *in-attention* mechanism. Prior to each attention layer, we sum expanded latent vector z_{pre} with every timestep of the previous hidden state, thus reinforcing its information throughout every stage of the decoding process. Therefor hidden state $H \in \mathbb{R}^{t \times d}$ at layer i is calculated as:

$$H_i = \text{SelfAttention}(H_{i-1} + x_{\text{pre}}) \tag{5}$$

The final hidden state is then passed through a feed-forward layer, which has weights tied to the embedding layers as first proposed in [23]. The decoder minimizes the negative log likelihood (NLL) of the output sequence y_t when given prior tokens:

$$\mathcal{L}_{\text{recon}} = \sum_{t=1}^{T} \log p_\theta(y_t|y_{<t}, z) \tag{6}$$

Thus, the composer is optimized with the NLL reconstruction loss as well as the β-scaled regularization KL loss:

$$\mathcal{L}_{\text{composer}} = \mathcal{L}_{\text{recon}} + \beta\mathcal{L}_{\text{KL}} \tag{7}$$

3.2 The Performer

The Performer is separately tasked with computing tokens for **velocity** and **microshift** time values. Thus, it is able to transform a quantized MIDI score into one with expressive characteristics, rendering it more suitable for a variety of music production tasks. Whereas the composition task generally benefits from an autoregressive setup wherein each token is predicted sequentially, it has been demonstrated in [24, 25] that performance attributes can be predicted in a bi-directional manner.

We utilise a framework comparable to the masked token prediction task of BERT [26]. During training and inference, we replace the input performance tokens denoting velocity and microtiming with a single `[MASK]` token. The tokens related to pitch, timeshift, and duration are left unmodified. The model is a standard transformer encoder as per [2], with sinusoidal positional embeddings, layer normalization and a final feedforward layer that has weights tied to the initial embedding layer.

During training the model is tasked to replace each `[MASK]` token with an appropriate velocity and microshift value, with cross-entropy loss used exclusively on the masked tokens. We perform the masking operation on 100% of performance tokens. At inference time, we manually mix tokens between the original source and model predictions, thus ensuring the original pitch, timeshift and duration values are maintained.

4. EXPERIMENTAL SETUP

4.1 Composer Ablations

By training several composer models, we aim to understand the relationship between various degrees of KL regularization and the decoded sequence's similarity to the input. Each model is trained on 4-bar segments of the full Lakh-MIDI dataset [27]. The models all have 12 layers and 8 heads in both the encoder and decoder, with a latent dimensionality d_z of 128 and hidden dimension d of 512. The *Full-KL* model was trained with a KL regularizer $\beta = 1.0$ and free bit $\lambda = 0.15$. The *Balanced-KL* model was trained with $\beta = 0.3$ and $\lambda = 0.25$. In both instances the KL regularization was applied with cosine cyclical annealing [28] every 10,000 steps. We initially keep $\beta = 0.0$ for the first 25,000 steps, and then linearly raise it to the maximum value over the proceeding 25,000 steps. Finally, the *No-KL* model had a $\beta = 0.0$ (no regularization), thus allowing the encoder to exclusively optimize against reconstruction quality.

Objective Evaluations : We generate a single variation for 500 files from the test set for each model, utilising greedy decoding to remove any sampling logic from the evaluation framework. For each sample, we calculate the similarity in **pitch distribution**, **onset locations** and **durations**:

$$similarity(x^a, x^b) = 100\frac{\langle x^a, x^b \rangle}{||x^a||||x^b||} \tag{8}$$

wherein $x \in \mathbb{Z}^t$ is a discrete vector of t attributes; in the case of **pitch** we set $t = 128$ to capture the full MIDI note range, and for both **onset location** and **duration** $t = 64$, representing the nearest 16th value in a 4-bar pattern. Finally, we report **Absolute Similarity**, the percentage of notes that have *identical* characteristics (pitch, onset, duration) between both sequences.

4.2 Performer Fidelity

Two performer models are trained with separate datasets to measure their capacity to model the unique expressive characteristics of a given training set. Both models are trained with 12 layers and heads, a hidden dimensionality of 768, and a dropout of 10%. One model is trained on the classical MusicNet dataset [29], and the other (referred to as *HipHop*) is trained on a proprietary hip-hop dataset. In both cases, we train on approximately 10,000 4-bar excerpts. Each dataset contains polyphonic data with differing expressive patterns of velocities and microtiming.

We randomly extract 2,000 polyphonic 4-bar patterns from the Lakh-MIDI dataset [27] and generate expressive tokens from both models. We subsequently measure the velocity and microtiming distributions of both the generations as well as the two original training datasets. **Velocity** distribution is represented as $v \in \mathbb{Z}^{128}$, a vector representing the number of occurences of each velocity value. For each note, **microtiming** is calculated as a percentage deviation from the nearest 16th note, with +/-50% denoting the halfway point to/from the adjacent 16th time index. The total distribution of **microtiming** deviations in a given sequence is thus represented with vector $mt \in \mathbb{Z}^{100}$.

For both velocity and microtiming, we compare the distributions of both model's predictions against the MusicNet and HipHop datasets. In **Table 4** we report the KL divergence, as well as the absolute difference in the mean and standard deviation for these distributions. In each metric, a lower value indicates higher degree of similarity between the model's predictions and the original dataset's expressive characteristics.

4.3 User Study

We conduct a thorough user study to achieve a qualitative understanding of our model's performance, comparing to different external baselines and versions due to hyperparameter settings. All the audio samples that users heard were 4 bar MIDI files voiced through the same Piano VST. In Part A of the user study, 25 human evaluators listened to 5 seed melodies and 4 alternative variations for each of them, coming from the *No-KL*, *Balanced-KL* and *Full-KL* versions of our proposed model, and additionally a *Placebo* melody which was randomly selected to be in the same key and scale as the input but had no relation to it. This was done to confidently ground the musical understanding of our human evaluators. Overall, 44% of our evaluators identified themselves as "Novice : I have little to no experience making music", 32% as "Amateur : I love making music for fun", and 24% as "Professional : I regularly make music in a professional capacity".

Model	Pitch Sim.(%)	Onset Sim.(%)	Duration Sim.(%)	Absolute Sim.(%)	Human Eval(1-5)
Full-KL	71.01	77.99	88.65	9.44	1.94
No-KL	95.64	83.04	98.40	20.05	2.84
Balanced-KL	92.06	80.80	96.95	16.46	2.71
Placebo	-	-	-	-	1.22

Table 2. Objective and human-evaluation results from the ablation studies. Higher values indicate more similarity to the input's musical characteristics.

In Part B, the same 25 human evaluators also rated 3 unconditional generations from Cadenza, Anticipatory Music Transformer (AMT) [14] [1] and Figaro [15] models, which broadly represent the state-of-the-art in symbolic polyphonic generation. We randomly sampled 3 generations from publicly-available checkpoints of each model, all of which were trained on identical versions of the Lakh MIDI dataset [27], and present the results in **Table 3**. As Cadenza's composer requires a latent vector, we randomly sample from a Gaussian distribution for unconditional generations. For each sample, the evaluator was asked to rate between how musically appealing it sounded to them with 1 being the lowest and 4 being the highest score. In addition, they were also asked to select, in a binary choice, whether they thought the performance was generated by a human or computer.

Model (Params)	Musical Appeal Score ↑ (1-4)	'Human-like' Score ↑ (1-2)
AMT (360M)	**3.33**	1.57 (57.3%)
Figaro (87M)	2.73	1.57 (57.3%)
Cadenza (142M)	2.91	**1.72 (72.0%)**

Table 3. Human Evaluation Results for Model Quality. Percentages represent the fraction of modeling outputs that were selected by human evaluators when asked if it could have been created by a human.

5. RESULTS AND DISCUSSION

We discuss our experimental results to answer three high level questions - 1) provided an input MIDI sequence, how *musically related* are the Cadenza variations; 2) can the performer model tangibly improve expressivity; and 3) how appealing are the novel generations. We analyze our proposed scientific approach through quantitative and qualitative measures.

Both human and objective evaluations demonstrate in **Table 2** that training the composer with varying degrees of KL regularization has noticeable impacts on the balance between between recall and variety. Provided a musical idea as input, the *No-KL* model will produce outputs nearly identical to the input melody. Alternatively, the *Balanced-KL* model will produce outputs that are related, yet altered enough to provide new sources of inspiration. In many cases, the *Full-KL* model will produce entirely unrelated outputs, as a result of the encoder's heavy focus on regularizing the latent vectors. Since a score of 4.0 for a generation would be considered identical we can infer that the ideas generated by *No-KL* and *Balanced-KL* are roughly 70% related to the input. This aligns with the quantitative results, which consistently show a negative correlation between the KL regularization and input/generation similarity metrics. As such, our framework is demonstrated to consistently generate variations that are perceptually relevant to, yet distinct from, the input musical idea.

[1] Specifically, the music-medium-800k checkpoint

Model (Metric)	KL		Mean Δ		Std Dev Δ	
	Train	Opposite	Train	Opposite	Train	Opposite
HipHop (Velocity)	**1.68**	3.63	**1.81**	16.83	**1.74**	9.04
HipHop (Microtiming)	**0.66**	2.64	0.05	0.05	**0.00**	0.14
MusicNet (Velocity)	**3.17**	11.57	**1.95**	13.06	**1.20**	12.00
MusicNet (Microtiming)	**0.07**	3.17	**0.02**	0.13	**0.01**	0.15

Table 4. Objective results on the Performer fidelity evaluations.

In **Table 4**, we report results from the Performer Fidelity quantitative study. In both MusicNet and HipHop models, the distributions of predicted velocities and microtimings are consistently closer to that of their respective training datasets. We can therefore infer that the performer model, in conjunction with our newly proposed PerTok tokenizer, is capable of accurately learning the patterns of expressive characteristics from a comparatively small dataset.

In **Table 3** we compare Cadenza to AMT [14] and Figaro [15] on the task of novel generations. Our model, although comparable to its competitors on musical appeal, comes in second to the AMT. However, Cadenza comprehensively outperforms other models by 14.7% in the *human-like* expressivity ratings. We note that our framework is highly adaptable, in that both composer and performer models could be replaced with any type of sequential network. Theoretically, one could further improve the unconditional generation quality by replacing our VAE-based composer model with a decoder-only model, similar to that of AMT.

6. CONCLUSION

We introduced a multi-stage generative framework which allows for both variation and novel generation tasks, maintaining a competitive quality of composition and setting a new state-of-the-art of expressive characteristics. Our proposed tokenizer is used to create expressive symbolic sequences while effectively reducing vocabulary size and sequence length. We invite readers to our model page [2] where we showcase its fidelity in generating outputs for polyphonic, monophonic, bass and drum instruments.

In particular, our performer model in conjunction with the new tokenization method led to a quantifiable increase in listener's perceptions of the expressivity in the generations. These results were achieved with relatively tiny datasets, paving the way for further collaborations with artistic communities. Future research directions include exploring *controllability*, as well as further improvements in the domain of novel generation.

[2] Access code and demonstrations on our model page here. PerTok tokenizer is available as part of MidiTok library here.

7. ETHICS STATEMENT

As generative AI technology advances rapidly, it is crucial to address the implications of these developments in the generative domain. Concerns such as perpetuating cultural biases, undermining artists' financial opportunities, and using data without proper consent require urgent attention and dialogue within research communities. When developing new models, we must carefully consider both their intended applications and potential impacts.

Our research involves deep collaboration with artists to understand their motivations and needs, ensuring our efforts benefit the creative communities we serve. For instance, our new framework, designed for the MIDI symbolic domain, focuses on enhancing artists' tools with features that inspire creativity rather than replacing the artists. We also deliberately chose to work with smaller models, which helps minimize data requirements. This strategy promotes fair data agreements and increases the chances of fairly compensating musicians, thus fostering sustainability in creative industries and prioritizing ethical responsibility, especially in creative domains.

8. REFERENCES

[1] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "Roformer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127063, 2024.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *CoRR*, vol. abs/1706.03762, 2017. [Online]. Available: http://arxiv.org/abs/1706.03762

[3] Y. Huang and Y. Yang, "Pop music transformer: Beat-based modeling and generation of expressive pop piano compositions," in *MM '20: The 28th ACM International Conference on Multimedia, Virtual Event / Seattle, WA, USA, October 12-16, 2020*, C. W. Chen, R. Cucchiara, X. Hua, G. Qi, E. Ricci, Z. Zhang, and R. Zimmermann, Eds. ACM, 2020, pp. 1180–1188. [Online]. Available: https://doi.org/10.1145/3394171.3413671

[4] N. Fradet, J.-P. Briot, F. Chhel, A. El Fallah Seghrouchni, and N. Gutowski, "MidiTok: A python package for MIDI file tokenization," in *Extended Abstracts for the Late-Breaking Demo Session of the 22nd International Society for Music Information Retrieval Conference*, 2021. [Online]. Available: https://archives.ismir.net/ismir2021/latebreaking/000005.pdf

[5] G. Hadjeres and L. Crestel, "The piano inpainting application," *CoRR*, vol. abs/2107.05944, 2021. [Online]. Available: https://arxiv.org/abs/2107.05944

[6] C. Hawthorne, A. Stasyuk, A. Roberts, I. Simon, C.-Z. A. Huang, S. Dieleman, E. Elsen, J. Engel, and D. Eck, "Enabling factorized piano music modeling and generation with the MAESTRO dataset," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=r1lYRjC9F7

[7] S. Oore, I. Simon, S. Dieleman, D. Eck, and K. Simonyan, "This time with feeling: Learning expressive musical performance," *CoRR*, vol. abs/1808.03715, 2018. [Online]. Available: http://arxiv.org/abs/1808.03715

[8] A. Maezawa, K. Yamamoto, and T. Fujishima, "Rendering music performance with interpretation variations using conditional variational rnn." in *ISMIR*, 2019, pp. 855–861.

[9] I. Borovik and V. Viro, "Scoreperformer: Expressive piano performance rendering with fine-grained control." in *ISMIR*, 2023, pp. 588–596.

[10] S.-L. Wu and Y.-H. Yang, "Compose & embellish: Well-structured piano performance generation via a

two-stage approach," in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.

[11] Y.-J. Shih, S.-L. Wu, F. Zalkow, M. Muller, and Y.-H. Yang, "Theme transformer: Symbolic music generation with theme-conditioned transformer," *IEEE Transactions on Multimedia*, 2022.

[12] H. H. Tan and D. Herremans, "Music fadernets: Controllable music generation based on high-level features via low-level feature modelling," *arXiv preprint arXiv:2007.15474*, 2020.

[13] S.-L. Wu and Y.-H. Yang, "Musemorphose: Full-song and fine-grained piano music style transfer with one transformer vae," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 1953–1967, 2023.

[14] J. Thickstun, D. Hall, C. Donahue, and P. Liang, "Anticipatory music transformer," *arXiv preprint arXiv:2306.08620*, 2023.

[15] D. von Rütte, L. Biggio, Y. Kilcher, and T. Hoffman, "FIGARO: generating symbolic music with fine-grained artistic control," *CoRR*, vol. abs/2201.10936, 2022. [Online]. Available: https://arxiv.org/abs/2201.10936

[16] J. Ens and P. Pasquier, "MMM : Exploring conditional multi-track music generation with the transformer," *CoRR*, vol. abs/2008.06048, 2020. [Online]. Available: https://arxiv.org/abs/2008.06048

[17] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv preprint arXiv:1312.6114*, 2013.

[18] D. P. Kingma, T. Salimans, R. Jozefowicz, X. Chen, I. Sutskever, and M. Welling, "Improved variational inference with inverse autoregressive flow," *Advances in neural information processing systems*, vol. 29, 2016.

[19] J. Lucas, G. Tucker, R. Grosse, and M. Norouzi, "Understanding posterior collapse in generative latent variable models," 2019. [Online]. Available: https://openreview.net/forum?id=r1xaVLUYuE

[20] S. Zhao, J. Song, and S. Ermon, "Infovae: Information maximizing variational autoencoders," *CoRR*, vol. abs/1706.02262, 2017. [Online]. Available: http://arxiv.org/abs/1706.02262

[21] T. Ucar, "Bridging the ELBO and MMD," *CoRR*, vol. abs/1910.13181, 2019. [Online]. Available: http://arxiv.org/abs/1910.13181

[22] A. B. Dieng, Y. Kim, A. M. Rush, and D. M. Blei, "Avoiding latent variable collapse with generative skip models," in *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and M. Sugiyama, Eds., vol. 89. PMLR, 16–18 Apr 2019, pp. 2397–2405. [Online]. Available: https://proceedings.mlr.press/v89/dieng19a.html

[23] O. Press and L. Wolf, "Using the output embedding to improve language models," *arXiv preprint arXiv:1608.05859*, 2016.

[24] B. Haki, M. Nieto, T. Pelinski, and S. Jordà, "Real-Time Drum Accompaniment Using Transformer Architecture," in *Proceedings of the 3rd International Conference on on AI and Musical Creativity*. AIMC, Sep. 2022. [Online]. Available: https://doi.org/10.5281/zenodo.7088343

[25] J. Lenz, "Disentangle and deploy: Generative rhythmic tools for musicians," Master's thesis, Pompeu Fabra University, 2023, available at https://zenodo.org/records/8380515.

[26] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," *CoRR*, vol. abs/1810.04805, 2018. [Online]. Available: http://arxiv.org/abs/1810.04805

[27] C. Raffel, "Learning-based methods for comparing sequences, with applications to audio-to-midi alignment and matching," Ph.D. dissertation, Columbia University, 2016.

[28] H. Fu, C. Li, X. Liu, J. Gao, A. Celikyilmaz, and L. Carin, "Cyclical annealing schedule: A simple approach to mitigating kl vanishing," *arXiv preprint arXiv:1903.10145*, 2019.

[29] J. Thickstun, Z. Harchaoui, and S. M. Kakade, "Learning features of music from scratch," in *International Conference on Learning Representations (ICLR)*, 2017.